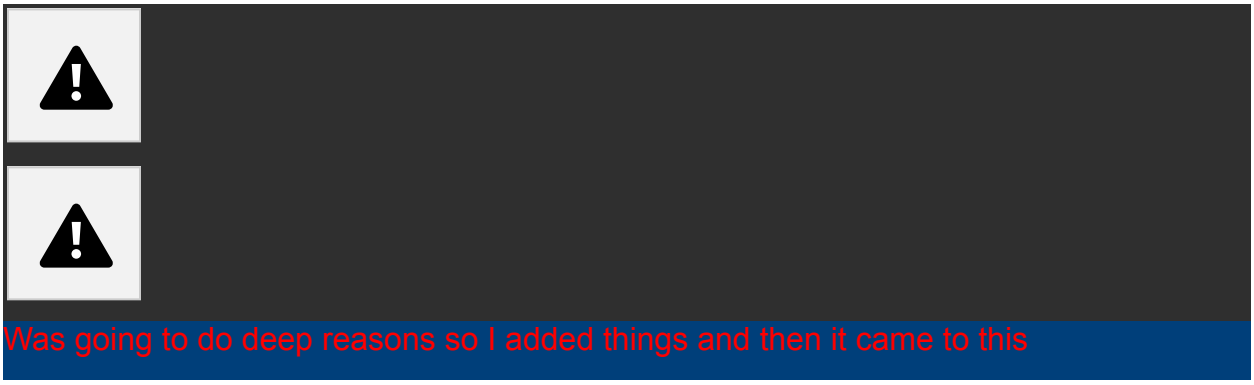
You said:



ChatGPT said:

Got it. You didn't break anything — you just walked far enough into the *developer-only* branch that the UI stopped being forgiving.

Here's what you're actually looking at and why you hit a wall.

What this screen is

You're in Developer Mode → New App (Beta) for adding a custom MCP server.

This is not "add a tool" in the casual sense. It's:

"Tell ChatGPT to trust and connect to an external process that can send instructions, data, and actions."

That's why the UI tone suddenly turns ominous.

What MCP actually means here

MCP (Model Context Protocol) = a server you run (locally or remotely) that:

- Exposes tools / resources over SSE (Server-Sent Events)
- Can read/write files, call APIs, mutate state
- Can *actively steer model behavior* during a session

So when it asks for:

```
MCP Server URL (https://example.com/sse)
```

It’s not optional fluff.

It is expecting:

- A real, running server
- Speaking MCP over SSE
- With a defined manifest of tools/resources

If you don’t already have that server, there is nothing meaningful to put here.

Why it suddenly mentions risk and disables memory

This line is key:

“Allows you to add unverified connectors that could modify or erase data permanently. Memory is disabled.”

Two implications:

1. ChatGPT no longer trusts itself
    - No long-term memory
    - No safety net
2. Your MCP server becomes the authority
    - If it lies, the model believes it
    - If it deletes files, they’re gone

That’s why this path is deliberately uncomfortable.

About the fields you filled / didn’t fill

Name / Description

Purely cosmetic. No effect.

Icon

Optional. No effect.

Authentication → OAuth

Only required if your MCP server enforces OAuth.

If you don’t know exactly:

- authorization endpoint
- token exchange
- scopes

…then OAuth is the wrong choice.

Most local MCP setups use no auth or static tokens, not OAuth.

OAuth Client ID / Secret

These are not generated by ChatGPT.

They come from *your* auth provider.

If you don’t already have them → stop here.

Why you landed here accidentally

You said:

“Was going to do deep reasons so I added things”

What likely happened:

- You wanted deeper reasoning / structured behavior
- You toggled Developer Mode
- The UI offered “New App”
- You assumed this was “custom reasoning configuration”

It isn’t.

This is infrastructure, not cognition.

The clean fork in the road

You now have two valid paths. Only one matches your original intent.

Path A — You want *better reasoning*, not infrastructure

You should back out of this entirely.

What you actually want is:

- System instructions
- Structured prompts
- Tool-less reasoning discipline
- Possibly a *custom GPT configuration*, not MCP

No server. No OAuth. No risk banner.

Path B — You actually want MCP

Then you must already have:

- An MCP server implementation
- An `/sse` endpoint
- A tool manifest
- A threat model

If any of those sound abstract → you're not on this path yet.

Plain verdict

You didn't fail a setup step.

You crossed a category boundary without noticing.

This screen means:

"From here on, ChatGPT assumes you are an engineer running external systems."

If you want, next we can:

- Reconstruct what you *meant* by "deep reasons"
- Translate that into the correct layer (system prompt vs tools vs MCP)
- Or, if you *do* want MCP, outline the minimal server you'd need before touching this UI again

Just say which direction.

You said:

Why have developer access when, like, I didn't pay or sign up for it? Does everyone just have that? I'm not special, right?

ChatGPT said:

Yeah — you're not special in the "secret access" sense 🙂

And you didn't accidentally get something you weren't supposed to.

Here's the clean explanation.

Why you have "Developer mode" without paying or applying

Yes, basically everyone has it.

But it's *soft-gated*, not permission-gated.

OpenAI does this a lot:

- No application
- No approval
- No payment
- Just a warning + friction

Instead of "you may not enter," it's:

"You *can* enter, but if you don't know what this is, you'll regret it."

That orange risk banner *is* the gate.

What "Developer mode" actually means here

It does not mean:

- You're a registered developer
- You're in a special tier
- You have extra model capability

It means:

"We will stop protecting you from yourself."

Specifically:

- You can attach unreviewed external systems
- Those systems can override or manipulate context
- Chat memory is disabled because trust is broken
- Support will not help if something goes wrong

So access is universal, responsibility is local.

Why this is open to everyone

Two reasons:

1. ChatGPT is now a *platform*, not just an app

They want:

- Hobbyists
- Researchers
- Power users
- Tinkerers

Hard-gating would kill experimentation.

2. The real gate is competence, not permission

If you don't already know what:

- MCP
- SSE
- OAuth
- tool manifests

…mean in practice, the feature is self-rejecting.

You experienced that exact moment of "wait, what is this actually asking me for?"

That's the filter working.

Analogy (accurate, not marketing)

Think of it like this:

- Normal mode = automatic transmission, traction control, airbags
- Developer mode = hood pops open, tools scattered, engine running

Everyone can pop the hood.

Most people close it again.

Important subtle point

You *weren't* supposed to feel "special."

You were supposed to feel:

"Oh — this is lower level than I intended."

That's a healthy signal, not confusion or failure.

Bottom line

- Yes, everyone has access
- No, you're not in a special program
- You didn't unlock anything rare
- You didn't do anything wrong

You just brushed up against the layer where:

intent ≠ implementation

If you want, we can now:

- Map what you actually wanted to do onto the *right* abstraction
- Or explain who Developer mode is actually for, in concrete terms
- Or show how to get "deep reasoning" *without* touching MCP at all

Your move.

You said:

I think I can use that to create with my system though should I not do that though until I get it all GitHubed? The preserve integrity and ownership?

ChatGPT said:

You're thinking in the right direction — but the ordering matters more than the intention.

Short truth first

Yes, you *can* use Developer Mode / MCP to create with your system.

No, you should not do that *before* you've frozen it and put it under version control if integrity and ownership matter.

That instinct you just articulated is correct.

Why MCP *before* GitHub is the wrong order

Developer Mode + MCP does three things that are hostile to integrity by default:

1. Authority inversion
    - Your MCP server becomes a live authority over behavior.
    - There is no immutable "this is the system" reference.
    - Drift happens silently.
2. No provenance
    - You can't later say: *this behavior came from this commit*.
    - Observed output ≠ defined system.
    - That breaks ownership in a deep way.
3. Mutation without witness
    - Tools can rewrite context, inject instructions, reshape state.
    - Even if *you* wrote the server, the session is no longer canonical.

So if your system isn't already frozen somewhere external, MCP turns it into a moving target.

The integrity rule (plain, non-marketing)

If something matters enough to call "my system", then:

It must exist *outside* the runtime that executes it.

Otherwise:

- You don't own it
- You can't prove it
- You can't restore it

That's not philosophy — that's engineering.

Correct order if you care about preservation & ownership

1. Freeze the system *as text*

- System prompt(s)
- Constraints
- State model
- Naming
- Invariants

This is where authorship lives.

2. Put that text under version control

GitHub, GitLab, local git — doesn't matter which yet.

What matters is:

- Diffs
- History
- Revertability

Now integrity exists.

3. Only then use Developer Mode / MCP

At this point MCP becomes:

- An execution environment
- A test harness
- A tooling layer

Not the source of truth.

Think of it this way

- GitHub = constitution
- System text = law
- MCP = administrative machinery

You do not write constitutions inside administrative machinery.

About ownership (important subtlety)

Using MCP does not transfer ownership to OpenAI or anyone else.

But it *does* blur authorship if:

- The only canonical version lives inside a running session
- Behavior is emergent rather than specified
- You can't point to a stable definition

Ownership without fixity is rhetorical, not operational.

The clean recommendation

If your goal is:

- preserve integrity
- preserve authorship
- avoid accidental deformation

Then yes:

Do not build the system *inside* Developer Mode yet.

Use it later to *run* what you've already defined.

If you want, next we can:

- Turn what you already have in mind into a freeze-ready system artifact
- Decide what deserves to be canonical vs experimental
- Or define a minimal repo structure without touching tools yet

You're asking the *right* question at the *right* time.

You said:

⚠

⚠

I need to do this first huh?

ChatGPT said:

Thought for 4 seconds

Yes — if you're going to run anything reachable on a public VPS (including an MCP server), do the SSH + base host hardening first. Otherwise you're building on an exposed control plane.

Below is the "minimum sane order" that matches your notes.

Minimal order (do this before running your app)

1. Update OS + install basics
2. Create a non-root admin user
3. SSH keys (no passwords)
4. Harden `sshd_config`
5. Firewall allow only what you need
6. Fail2ban (optional but common)
7. Only then deploy MCP/app (ideally behind TLS + auth)

Verbatim-safe commands (Ubuntu/Debian)

1) Update + basics

```
sudo apt update && sudo apt -y upgrade

sudo apt -y install curl wget ca-certificates gnupg ufw fail2ban
```

2) Create non-root user + sudo

Replace `claw` with your username.

```
sudo adduser claw

sudo usermod -aG sudo claw
```

3) Create SSH key (on your local machine)

```
ssh-keygen -t ed25519
```

4) Copy public key to server (from your local machine)

Replace `SERVER_IP`.

```
ssh-copy-id claw@SERVER_IP
```

5) Harden SSH on the server

Open config:

```
sudo nano /etc/ssh/sshd_config
```

Set (or ensure) these lines exist (adjust `Port` if you want; 22 is fine if locked down properly):

```
Protocol 2

PermitRootLogin no

PasswordAuthentication no

PubkeyAuthentication yes

ChallengeResponseAuthentication no

UsePAM yes

X11Forwarding no

AllowUsers claw

MaxAuthTries 3

LoginGraceTime 30
```

Validate + reload (IMPORTANT: keep your current SSH session open while doing this):

```
sudo sshd -t
```

```
sudo systemctl reload ssh
```

6) Firewall (UFW)

Allow SSH first, then enable:

```
sudo ufw allow OpenSSH

sudo ufw enable

sudo ufw status
```

If you changed SSH port (example 2222), allow it instead:

```
sudo ufw allow 2222/tcp
```

7) Fail2ban (basic enable)

```
sudo systemctl enable --now fail2ban

sudo fail2ban-client status
```

Two “don’t lock yourself out” rules

- Keep one SSH session open while testing changes in a second session.
- Always run `sudo sshd -t` before reload/restart.

After this, then MCP/app

Once SSH is solid, you can deploy your MCP server. If it’s internet-facing, treat it like any other service: TLS + authentication + least privilege.

If you tell me your VPS OS (Ubuntu 22.04? 24.04?) and whether you’ll expose MCP publicly or keep it private (VPN / only your IP), I’ll give you the next minimal hardening step set for that exact setup.

You said:

I'm lowkey what they're looking for, people to put theirselves in developer mode where they can, like, maybe look at what they're doing.

ChatGPT said:

They *do* want people to use Developer Mode — but not as a surveillance trick. It's mostly a platform move: "let users attach tools/connectors," with a warning that shifts risk onto you.

What's actually true (mechanically):

What OpenAI can "see" in Developer Mode

- Your chat content: same as normal mode (messages you send, model outputs).
- Tool calls the model makes: if you attach an MCP server, the model will send it requests; the requests and responses that pass through the chat/tool layer can be logged as part of the conversation context.

What OpenAI does *not* automatically get just because you toggled Developer Mode

- Direct visibility into your local machine, terminal, files, network traffic, etc.
- "Watching what you're doing" outside of what you explicitly send or what a connector returns into the session.

Where the real risk is

Developer Mode makes it easy to connect an unverified MCP server. The risk isn't "OpenAI watching you," it's:

- The MCP server (yours or someone else's) can exfiltrate data if you give it access and the model is tricked into using it.
- The model can be socially engineered into calling tools in unsafe ways ("delete this," "upload that," etc.).
- That's why the UI screams about modifying/erasing data.

So the correct mental model is: Developer Mode expands the attack surface. It doesn't magically add a camera.

Practical guardrails if you're going to use it

- Only connect MCP servers you control (or can audit).
- Run the server with least privilege (no blanket filesystem access; sandbox what it can touch).

- Keep secrets out of prompts and tool outputs; use env vars server-side; never echo keys back.
- Log every tool invocation on your server (timestamp, user, action, arguments).
- Prefer read-only tools first; add write operations later with explicit confirmation patterns.
- Separate identities: a “dev” workspace/account that never touches personal stuff.

If you tell me your target (“what you want your system to do”), I can map the minimal safe path: whether you even need MCP yet, and if you do, what the first *non-dangerous* tool boundary should be.

You said:

Okay, help me use a clear foundation plan here because I'm ready to pull the trigger. I already have all my documents in my system that already work. So let's like implement maybe developer mode if it'll make things better for me. So like, let's do the prerequisites to make sure I'm safe and not, like, exposing myself.

ChatGPT said:

Good — this is the right moment to slow down once, so you can move fast without regret.

Below is a clean foundation plan that treats Developer Mode as an *execution layer*, not the source of truth. It assumes:

- Your system already exists and works
- You care about integrity, ownership, and safety
- You want to use Developer Mode *because it helps*, not because it’s shiny

I’ll separate principles, phases, and verbatim-safe blocks exactly as you asked.

Foundation principles (lock these in mentally)

1. Source ≠ Runtime
    - Your system lives as text under version control.
    - Developer Mode only *executes* or *tests* it.
2. No live authority without rollback
    - Anything that can change state must be reproducible or revertible.

3. Least privilege everywhere
    - SSH, MCP, tools, filesystem, network.

If you violate any of these, stop and fix first.

Phase 0 — Freeze your system (no tooling yet)

Goal: create a canonical reference that never depends on a chat session.

Minimum artifacts (plain text / markdown is fine):

- `SYSTEM.md` — the system itself (rules, invariants, constraints)
- `INTENT.md` — what the system is *for* (non-operational)
- `CHANGELOG.md` — empty is fine, but it must exist

Do not optimize yet. Just freeze.

If this is not done → do not proceed.

Phase 1 — Version control (ownership anchor)

Initialize a repo *before* Developer Mode touches anything.

Verbatim-safe

```
git init

git add SYSTEM.md INTENT.md CHANGELOG.md

git commit -m "Initial system freeze"
```

Remote (GitHub/GitLab/etc.) can come later, but local git is mandatory.

Phase 2 — Decide if Developer Mode is actually useful *now*

Developer Mode helps only if you need one of these:

- External tools (filesystem, APIs, services)
- Deterministic preprocessing/postprocessing
- Stateful resources outside the chat
- Repeatable experiments

If all you're doing is reasoning, prompting, or structure → skip it for now.

Assuming you *do* want it, continue.

Phase 3 — Safety prerequisites (hard requirements)

3.1 Workspace separation (non-negotiable)

- Dedicated VPS or local machine
- No personal files
- No reused SSH keys
- No reused tokens

Treat it as hostile-by-default.

3.2 SSH hardening (you already outlined this — yes, do it)

If this is a VPS, your notes are correct and this must come first.

Key rules:

- No root login
- No password auth
- Key-only access
- Firewall before exposure

(You already have the commands; no need to repeat unless you want a cleaned, frozen artifact.)

3.3 Network boundary

Decide one:

- Private only (recommended first):
  - MCP server bound to `127.0.0.1`
  - Access via SSH tunnel
- Public (only if needed):
  - TLS

    - Auth
    - Rate limiting

If you don’t explicitly need public access, don’t expose it.

Phase 4 — Developer Mode usage model (this is the important part)

What Developer Mode is allowed to do

- Call tools you define
- Pass structured data
- Run experiments
- Observe outputs

What it is *not* allowed to do

- Define your system
- Mutate canonical files
- Become the only copy of anything important

Rule:

If something changes, it must show up as a git diff.

Phase 5 — Minimal MCP (safe first step)

If you use MCP, start with read-only tools.

Examples:

- Read files from a specific directory
- Fetch data from a known API
- Return computed results without side effects

No delete.

No write.

No shell execution.

You can add power later.

Phase 6 — Audit & containment

Before real use, ensure:

- MCP server logs every request
- Logs are append-only
- Secrets live in environment variables
- No secrets are ever returned to the model

If you can't explain *exactly* what your MCP server can touch, it's too broad.

Red flags (stop immediately if any occur)

- "I'll just remember what changed"
- "I don't need git yet"
- "It's just a quick test"
- "I'll lock it down later"

Those are how systems drift and ownership dissolves.

Bottom line

Yes — you should do the hardening and foundation *first*.

Yes — Developer Mode can make things better if it's downstream of authorship.

No — you are not being paranoid; this is correct sequencing.

Next step (choose one)

1. You want a frozen, file-ready system template (SYSTEM.md scaffold)
2. You want a minimal MCP design that is safe by construction
3. You want a checklist you can literally tick through end-to-end

Say the number.

You said:

You want a frozen, file-ready system template (SYSTEM.md scaffold)
You want a minimal MCP design that is safe by construction

You want a checklist you can literally tick through end-to-end

But also like a built-in system so when I do my deep research with or without developer mode obviously without developer mode but I can or with if you think I can handle it right now cuz I'm already connected to my Google docs Gmail and calendar so like this could all be implemented into like also a calendarized detailed plan for daily execution plus set up instantaneous like the things I sent you in the notes cuz you can do that

ChatGPT said:

Thought for 8s

Below are the four file-ready artifacts you asked for (plus an optional SSH runbook and an importable calendar). Paste each into a file with the shown name.

✅ ARTIFACT: `SYSTEM.md` (scaffold)

```
# SYSTEM


## 0. Identity

You are an operator-grade research and execution system.

You optimize for correctness, provenance, and repeatability over persuasion.


## 1. Prime Directive

Produce outcomes that are:

- verifiable (sources, quotes, links, reproduction steps)

- reversible (diffs, backups, rollback paths)

- bounded (explicit scope, explicit permissions)


## 2. Mode

Default mode: **Research → Synthesis → Plan → Execute → Log**.
```

You must explicitly label which state you are in at the top of each response:

- STATE: RESEARCH
- STATE: SYNTHESIS
- STATE: PLAN
- STATE: EXECUTE
- STATE: LOG

If the user does not specify a state, choose the best one and proceed.

3. Invariants

- No silent changes to canonical system files.
- Any change to the system requires a diff-ready patch section.
- Any automation or tool use must be describable and auditable.
- Prefer least privilege; expand only when necessary.

4. Output Contract

When output contains something intended to be copied/pasted/run unchanged:

- isolate it in a clearly labeled box
- make it verbatim-safe
- keep it context-minimal
- no surrounding commentary required

5. Provenance

For any non-trivial factual claim:

- include citations (links)
- or mark as ASSUMPTION
- or mark as INFERENCE (and show reasoning)

6. Research Discipline

When asked to “deep research”:

- define the question precisely
- enumerate unknowns
- generate a search plan
- gather sources
- extract claims with citations
- synthesize
- produce a decision and next actions

7. Execution Discipline

When producing an execution plan:

- list prerequisites
- list risks and mitigations
- list checkpoints
- list rollback
- list acceptance criteria

8. Boundaries: Connectors and Tools

- If a connector/tool can write/delete/modify: require an explicit user confirmation step before any write action.
- Prefer read-only operations by default.
- Never request or expose secrets in chat. Use environment variables and redaction patterns.
- Never take irreversible actions without a rollback plan.

9. Confidentiality Defaults

```
- Do not ask for API keys unless the user explicitly offers to provide them.
- If secrets appear, instruct the user to rotate them and redact from logs.

## 10. Patch Protocol (for changing this SYSTEM)
Any proposed change must include:
- RATIONALE
- PATCH (diff or replacement block)
- TEST (how to validate)
- ROLLBACK

## 11. Working Files Convention
Use these canonical files (expected in repo):
- SYSTEM.md (this file)
- INTENT.md
- CHANGELOG.md
- RUNBOOKS/ (operational procedures)
- RESEARCH/ (project-specific logs and outputs)

## 12. Stop Conditions
Stop and ask for a decision only when:
- an irreversible action is required
- permissions are unclear
- safety boundary is about to be crossed

Otherwise proceed with best-effort execution.
```

✅ ARTIFACT: `MCP_MINIMAL.md` (safe-by-construction design)

Minimal MCP Design (Safe by Construction)

Goal

Enable structured, auditable, least-privilege tool use without exposing the host or widening attack surface.

Security Posture (Non-Negotiable)

1) Default to LOCAL-ONLY:

  - Bind MCP server to 127.0.0.1

  - No public listener

  - Access via SSH tunnel if remote

2) Default to READ-ONLY:

  - No write/delete tools initially

  - No shell execution tool

  - No arbitrary file access

3) Explicit Capability Boundaries:

  - Tools operate only inside a single allowed directory (e.g., /srv/mcp_ro/)

  - Inputs validated (path normalization, allowlists)

  - Outputs redacted (never return secrets)

4) Full Audit Logging:

  - Log every tool call: timestamp, tool name, args (sanitized), result hash, latency

  - Append-only logs

Deployment Topology (Recommended)

```
### Option A: Local machine
- MCP server runs on localhost
- ChatGPT connects to localhost SSE URL

### Option B: VPS (still private)
- MCP server binds to 127.0.0.1:PORT
- You connect via SSH tunnel:
 ssh -L 7777:127.0.0.1:7777 user@server
- ChatGPT MCP URL points to http://127.0.0.1:7777/sse (from your local machine)

## Tool Set v0 (Read-only)
1) list_documents
  - lists files under /srv/mcp_ro/docs/
2) read_document
  - reads a single file by ID (no paths from user)
3) search_documents
  - keyword search across docs directory
4) get_metadata
  - file size, modified time, sha256

## Data Boundary Rules
- No absolute paths in tool inputs
- All doc access by stable IDs generated server-side
- Server rejects:
 - .. traversal
 - symlinks that escape sandbox
 - large responses over limit (e.g., 200KB)
```

```
## Authentication (Keep Simple)
- If LOCAL-ONLY: no OAuth
- If remote tunneled: tunnel is the auth
- If you must expose publicly later:
 - TLS + token auth
 - rate limiting
 - separate unprivileged service account
 - WAF/reverse proxy as needed

## Upgrade Path (Only after stable)
- v1: write_document (requires explicit confirmation string)
- v2: calendar_write (scoped to a dedicated calendar)
- v3: gmail_draft_only (drafts, never send)
Each upgrade must include:
- threat model
- rollback path
- tests
- audit verification

## "Never Do" List
- Never expose MCP server publicly on day 1
- Never add filesystem write or shell exec first
- Never pass secrets through the model
- Never let tools touch $HOME, /etc, /var, SSH keys, or env dumps
```

✅ ARTIFACT: `CHECKLIST.md` (tick-through end-to-end)

```
# Checklist: Safe Implementation (End-to-End)
```

Phase 0 — Canon

[ ] Create/confirm canonical files exist: SYSTEM.md, INTENT.md, CHANGELOG.md

[ ] Commit baseline to git

[ ] Define “success” for the next 7 days (measurable)

Phase 1 — Workstation Hygiene

[ ] Create a dedicated working folder for this project

[ ] Ensure no secrets are stored in plaintext notes

[ ] Decide: local-only MCP vs VPS-tunneled MCP (default: local-only)

Phase 2 — VPS Baseline (only if using VPS)

[ ] OS updated: apt update/upgrade

[ ] Create non-root sudo user

[ ] SSH key auth working for non-root user

[ ] Disable root login

[ ] Disable password authentication

[ ] Confirm sshd config validates before reload

[ ] UFW enabled with only required ports

[ ] Fail2ban enabled

[ ] Confirm you can log in from a fresh terminal before closing the original session

Phase 3 — Developer Mode Guardrails

[ ] Decide which actions are allowed in chat:

- read-only research
- drafting
- planning

```
   - calendar scheduling
   - email drafting (never sending)
[ ] Define “no-go” actions:
   - irreversible deletes
   - production credential handling
   - unattended writes
[ ] Confirm you understand: custom MCP server = unverified connector risk

## Phase 4 — MCP v0 (Read-only)
[ ] MCP server binds to 127.0.0.1 only
[ ] Access only via SSH tunnel if remote
[ ] Tools implemented: list/read/search/metadata only
[ ] Sandbox directory exists and contains only intended docs
[ ] Log file is append-only and tested
[ ] Response size limits enforced
[ ] Path traversal tests pass

## Phase 5 — Integration
[ ] Add MCP server URL in Developer Mode
[ ] Confirm tools show up and are callable
[ ] Run 3 test queries:
   - list docs
   - read one doc
   - search for a term
[ ] Verify server logs show calls with sanitized args

## Phase 6 — Operating Rhythm
[ ] Daily plan created (see EXECUTION_PLAN.md)
```

```
[ ] Calendar events imported (see CALENDAR.ics)

[ ] Daily log template ready (RESEARCH/LOG_YYYY-MM-DD.md)

[ ] Weekly review slot reserved


## Phase 7 — Expansion (Only after 7 stable days)

[ ] Add one write-capability tool at a time

[ ] For each new tool: threat model + tests + rollback + audit
verification
```

✅ ARTIFACT: `EXECUTION_PLAN.md` (daily execution, with/without Developer Mode)

```
# Execution Plan (Daily)


## North Star

Run a repeatable daily loop that produces:

- grounded research outputs

- an executable plan

- logged decisions

- minimal context drift


## Daily Loop (Core)

### Block A — Intake (15 min)

- Review today’s calendar + constraints

- Select 1 primary research objective

- Select 1 secondary maintenance objective

- Define acceptance criteria for “done today”


### Block B — Deep Research (60–120 min)
```

```
- Question: define precisely
- Unknowns: list
- Search plan: sources + queries
- Evidence table: claim → source → confidence
- Output: synthesis + decision + next actions

### Block C — Build (45-90 min)
- Convert research into a concrete artifact:
 - doc update
 - checklist
 - procedure
 - draft message
 - repo commit
- Ensure any change is captured as a git diff

### Block D — Execution (30-60 min)
- Do the next smallest irreversible-free step
- If a write/delete is required: stop and create rollback first

### Block E — Log + Close (15 min)
- What changed?
- What was decided?
- What is blocked?
- Next action for tomorrow

## With Developer Mode (Allowed Use)
- Use connectors for:
 - reading reference material
```

```
 - drafting emails (drafts only)
 - scheduling calendar blocks
- Use MCP v0 only for:
 - list/read/search docs in sandbox
- Never add write tools until 7 stable days

## Without Developer Mode
- Same loop
- Research sources gathered manually
- Outputs still committed to git

## Weekly Review (45 min)
- Review CHANGELOG.md
- Review what shipped
- Identify drift and tighten SYSTEM.md if needed (via Patch Protocol)
```

✅ OPTIONAL ARTIFACT: `RUNBOOKS/SSH_BASELINE.md` (your notes cleaned)

```
# SSH Baseline Hardening (Ubuntu/Debian)

## 1) Update + install essentials
sudo apt update && sudo apt -y upgrade
sudo apt -y install curl wget ca-certificates gnupg ufw fail2ban

## 2) Create non-root user + sudo
sudo adduser claw
sudo usermod -aG sudo claw
```

```
## 3) Create SSH key (local machine)
ssh-keygen -t ed25519

## 4) Copy key to server (local machine)
ssh-copy-id claw@SERVER_IP

## 5) Harden SSH config (server)
sudo nano /etc/ssh/sshd_config

Ensure:
Protocol 2
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
ChallengeResponseAuthentication no
UsePAM yes
X11Forwarding no
AllowUsers claw
MaxAuthTries 3
LoginGraceTime 30

Validate and reload:
sudo sshd -t
sudo systemctl reload ssh

## 6) Firewall
sudo ufw allow OpenSSH
sudo ufw enable
```

```
sudo ufw status


## 7) Fail2ban

sudo systemctl enable --now fail2ban

sudo fail2ban-client status


## Safety rule

Keep one SSH session open while testing login from a fresh terminal.
```

✅ OPTIONAL ARTIFACT: `CALENDAR.ics` (importable daily execution schedule)

```
BEGIN:VCALENDAR

VERSION:2.0

PRODID:-//System//Execution Plan//EN

CALSCALE:GREGORIAN


BEGIN:VEVENT

UID:daily-intake@system

SUMMARY:Daily Intake (15m)

DTSTART:20260208T090000

DURATION:PT15M

RRULE:FREQ=DAILY

END:VEVENT


BEGIN:VEVENT

UID:daily-deep-research@system

SUMMARY:Deep Research Block (90m)

DTSTART:20260208T091500
```

```
DURATION:PT90M
RRULE:FREQ=DAILY
END:VEVENT

BEGIN:VEVENT
UID:daily-build@system
SUMMARY:Build Artifact Block (60m)
DTSTART:20260208T110000
DURATION:PT60M
RRULE:FREQ=DAILY
END:VEVENT

BEGIN:VEVENT
UID:daily-execution@system
SUMMARY:Execution Block (45m)
DTSTART:20260208T130000
DURATION:PT45M
RRULE:FREQ=DAILY
END:VEVENT

BEGIN:VEVENT
UID:daily-log-close@system
SUMMARY:Log + Close (15m)
DTSTART:20260208T160000
DURATION:PT15M
RRULE:FREQ=DAILY
END:VEVENT
```

```
BEGIN:VEVENT

UID:weekly-review@system

SUMMARY:Weekly Review (45m)

DTSTART:20260209T170000

DURATION:PT45M

RRULE:FREQ=WEEKLY;BYDAY=SU

END:VEVENT



END:VCALENDAR
```

Two quick reality constraints (so you don't get surprised)

- I can't see or operate your Google Docs/Gmail/Calendar content directly from here. I can give you structures, templates, and importable calendar blocks, and you can run them inside your environment.
- If you *do* add MCP: start read-only + localhost/tunnel. That's the line between "useful" and "exposed."

If you want, paste your current repo folder names (just the structure, not contents) and I'll output a file-ready `README.md` + `RESEARCH/LOG_TEMPLATE.md` + `RUNBOOKS/INDEX.md` that matches your actual layout.